

DIC1 GENIE INFORMATIQUE**Mini-Projet de POO****TITRE**

ANALYSE SYNTAXIQUE D'EXPRESSIONS ARITHMETIQUES

DATE DE PRESENTATION ET DE LIVRAISON

11/07/2026

GROUPES

Le mini projet peut être réalisé par groupes de cinq à six étudiants. Chaque étudiant devra être en mesure d'expliquer l'intégralité du code de l'application ainsi que les différents choix opérés (chaque mini projet devra être présenté).

LIVRABLES

Chaque groupe devra fournir:

- le code source clair et bien **documenté**,
- un rapport ou une page Web donnant accès à la documentation API de toutes vos classes.

DESCRIPTION

1 Introduction

Ecrire un analyseur syntaxique d'expressions arithmétiques

Soit la grammaire :

```
E -> S ;  
S -> P | P + S  
P -> T | T * P  
T -> C | ( S )  
C -> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

dans laquelle E est mis pour expression, S pour somme, P pour produit, T pour terme et C pour chiffre.

On va donner trois versions de ce programme. Dans les trois, toutes les méthodes d'analyse feront partie d'une même classe. Il n'est pas question de construire explicitement l'arbre syntaxique de la chaîne analysée. Le principe général est d'associer une méthode à chaque symbole non terminal de la grammaire : somme() associée à S, ...

erreurs de syntaxe, la dernière utilise les exceptions.

Entre la première et la seconde, la différence est seulement dans la gestion du débordement de la chaîne d'entrée.

Première version

- Ecrire une classe `Source`, contenant la chaîne de caractères à analyser et l'indice du caractère courant. Y définir les deux méthodes :
 - `premier()`, qui retourne le prochain caractère pris dans la chaîne à analyser, sans le supprimer ;
 - `suivant()`, qui incrémente le pointeur. Dans la première version, on ne se préoccupera pas de traiter le débordement de la chaîne.
- Ecrire une classe `Analyseur` dans laquelle on définira 5 méthodes :
 - `expression()`,
 - `somme()`,
 - `produit()`,
 - `terme()`,
 - `chiffre()`.

On créera dans cette classe un champ `Source`, pour accéder à la chaîne à analyser. La méthode `analyseur()` se contente d'appeler `expression()`, de vérifier la présence d'un point virgule, et d'écrire un message selon le résultat de l'analyse. Toutes les méthodes retourneront une valeur booléenne pour gérer les erreurs. Une erreur détectée par la méthode `chiffre()` par exemple sera remontée jusqu'à la méthode `analyseur()`, qui émettra le message d'erreur.

- Ecrire une classe `Principale` pour exécuter cet analyseur. Lui soumettre la chaîne `"2+3;"`.

seconde version

La règle `expression` semble inutile. Pour en voir l'intérêt, lancez l'analyse directement par `somme()` (dans `analyseur()`, remplacez `expression()` par `somme()`).

Essayez la chaîne `"2+3$4;"` dans laquelle un caractère `$` a été tapé à la place de `*`. Vous constatez que l'analyseur ne relève pas d'erreur ! Comment l'expliquez-vous ?

Autre essai à faire : analysez `"2+3"` (sans le point virgule). A la fin de l'analyse, il doit se produire une exception `StringIndexOutOfBoundsException`.

Ces deux essais montrent qu'il faut savoir où doit se trouver la fin de la chaîne à analyser, d'où l'intérêt du point virgule. Si l'utilisateur oublie le point virgule, le débordement doit être traité correctement. Vous modifierez la classe `Source` pour propager les erreurs de débordement (toujours avec des booléens).

Troisième version [A IGNORER]

Nous allons maintenant remplacer les booléens par les exceptions.

1. Modifiez la classe `Source` : les deux méthodes ne retournent rien (`void`), mais

jettent l'exception `Exception`.

2. Créer une classe `SyntaxException` qui étend `Exception` et redéfinit les deux constructeurs (celui sans paramètre, et celui avec une chaîne en paramètre) ; 3. Modifier la classe `Analyseur` en transformant les méthodes associées aux symboles non terminaux (`somme()`, `produit()` etc.) : elles ne retourneront plus de valeur booléenne, mais jetteront éventuellement ou propageront une `SyntaxException`, qui sera récupérée et traitée par la méthode `analyser()`.
4. Modifiez la classe `Principale` pour exécuter cet analyseur. Elle devra intercepter les deux sortes d'exceptions `Exception` (provenant de `Source`), et `SyntaxException`.

7 Interpréteur

Transformer le programme précédent pour en faire un interpréteur.

Lorsqu'on lui présente une expression, l'interpréteur émettra un message d'erreur si elle est incorrecte, et donnera sa valeur si elle est correcte. La modification du programme se fait en ajoutant une seule instruction à chaque méthode !

Pour faire les calculs, il faut gérer une pile (faire une classe `Pile`). Lorsqu'on trouve un chiffre, on empile sa valeur.

Généralement, chaque méthode doit empiler la valeur qu'elle calcule. Ainsi, dans la méthode `somme()`, on appelle `produit()`, qui empile la valeur du produit. S'il n'y a pas de + après, la valeur empilée du produit est la valeur de la somme, qui est donc déjà empilée. Sinon, on analyse une somme, qui empile son résultat. Il reste à dépiler les deux valeurs du sommet de la pile, à les ajouter, et à empiler le résultat.

Changez le nom de la méthode `analyser()` en `interpreteur()`.

Après l'appel de l'interpréteur, le résultat se trouve au sommet de la pile (il y est d'ailleurs seul). Il ne reste à la méthode `interpreteur()` qu'à écrire le résultat pris au sommet de la pile.

8 Compilateur

Transformer le programme obtenu pour en faire un compilateur.

Le compilateur consiste à retarder, en les écrivant dans un fichier (ou sur l'écran), les instructions qui composent l'interpréteur. Il suffit donc de les faire écrire au lieu de les exécuter.

Tout simplement, mettre les instructions qui utilisent la pile dans un `println()` !

